BDA Mini Project Report

City Problems – Mumbai Civic Issues Map

Submitted fulfilment of the requirements of the
BDA Mini Project Semester VII in degree of
Bachelor of Computer Engineering

BY

Ayan Parkar (63)
Sourabh Patel (64)
Mohammed Owaiz Shaikh (65)

Guide
Prof. Puja Sarage

Department Of Computer Engineering
Shree L. R. Tiwari College Of Engineering
Kanakia Park, Mira Road (E), Thane -401 107, Maharashtra.

2026-27

BDA Mini Project Report Approval

This project report entitled **“City Problems – Mumbai Civic Issues Map”** by **Mr. Ayan Parkar (63), Mr. Sourabh Patel (64) and Mr. Mohammed Owaiz Shaikh (65)** is approved for the Mini Project Semester VII project work of B.E CMPN at Shree L.R.Tiwari College of Engineering, in the academic year 2026-2027.

Internal Examiner

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

_____

Signature

Mr. Ayan Parkar (63)

_____

Signature

Mr. Sourabh Patel (64)

_____

Signature

Mr. Mohammed Owaiz Shaikh (65)

Date:

Place:

Table of Contents

1. Introduction

2. System Architecture

3. Implementation

Technologies Used

4. Results

5. Conclusion

6. References

1. Introduction

Mumbai's civic body, the Brihanmumbai Municipal Corporation (BMC), receives tens of thousands of citizen complaints every few months: potholes, uncollected garbage, blocked drains, water-supply faults, broken street lights, encroachment and more. Each complaint is a small record, but together they form a large, geo-tagged, time-stamped dataset that can show where the city is failing, how fast problems are fixed, and how complaint patterns change with seasons such as the monsoon.

City Problems is a Big Data Analysis (BDA) mini-project that turns this public data into an interactive map and a query workbench. About 97,000 cleaned complaints (March–October 2026) are shown as clustered markers, a heatmap and a ward-level choropleth. A query builder lets the user group, aggregate and filter the data (group-by, count, average, percentage, per-capita rate, HAVING-style thresholds) and shows the equivalent SQL, a chart and a table for every query.

1.1 Big data perspective

- **Volume —** 124,146 raw records were downloaded (112,086 general complaints + 12,060 pothole records); 96,999 cleaned records are analysed (~13 MB CSV) in the browser.
- **Variety —** Structured complaint rows (category, ward, status, dates, source channel), geospatial point data (latitude/longitude) and polygon data (26 ward boundaries with population).
- **Velocity —** The BMC MARG service is a live system that keeps receiving complaints and changing statuses; the project snapshot was fetched on 11 Oct 2026 and the pipeline can be re-run to refresh it.
- **Veracity —** Raw data contains placeholder and invalid coordinates; a cleaning pipeline removes these so that analytics are not distorted (Section 2.2).

1.2 Objectives

- Acquire a real, geo-tagged Mumbai civic-complaint dataset from a public source.
- Clean and integrate multiple layers into one analysis-ready schema.
- Visualise the issues on a map (markers, heatmap, ward choropleth).
- Provide flexible group-by / aggregate queries and eight ready-made preset queries.
- Keep the app fully static so it can be hosted without a backend.

2. System Architecture

2.1 Data source

The data comes from the public MARG ArcGIS REST service of BMC, the same service used by BMC's own Pothole QuickFix dashboard (https://marg.mcgm.gov.in/BMC_C_Pothole_Dashboard/). The endpoint is https://marg.mcgm.gov.in/server/rest/services/MARG_APP/MARG_Application_Prod/MapServer. Three layers are used:

- **Layer 0 COMPLAINTS_REGISTRATION —** 112,086 general citizen complaints with their own latitude/longitude.
- **Layer 16 POTHOLE_REGISTRATION —** 12,060 pothole records from Pothole QuickFix.
- **Layer 2 Ward_Boundary —** 26 ward polygons with a population field.

Licence note: BMC states no explicit open-data licence on this service. It is public, unauthenticated government data and is used here only for non-commercial academic purposes with attribution to BMC/MCGM. Permission from BMC should be obtained before any commercial reuse. Only non-personal fields were downloaded (no phone numbers, free-text descriptions, officer or contractor names, or photos). Other candidate datasets were rejected: the Kaggle "1.2M Complaints BMC" set is documented as synthetic, and no downloadable real Mumbai complaint CSV exists on data.gov.in.

2.2 Data pipeline (scripts/download_raw.py, scripts/prepare_data.py)

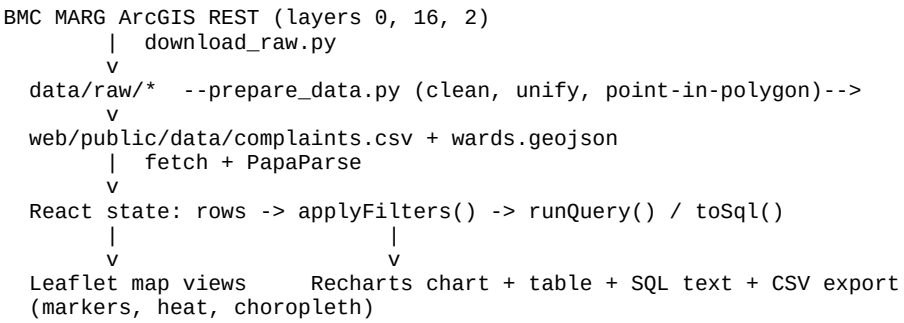
1. Download layers 0, 16 and 2 from the ArcGIS REST service (non-personal fields only).
2. Unify layers 0 and 16 into one schema: id, date, category, subcategory, ward, zone, status, lat, lon, res_days, source, dataset. Invalid pothole records become status Discarded.
3. Drop 550 rows with missing, projected or out-of-Mumbai coordinates.
4. Drop 14,546 rows that all sit on one placeholder point (19.07766, 72.87741), mostly the "1916" helpline channel and invalid potholes with no real location; the next busiest point holds only 132 rows, and keeping them would have made ward L look falsely huge.
5. Fill 58 rows with an unknown ward using point-in-polygon on the BMC ward boundaries (Shapely STRtree).
6. Derive a clean category (11 classes, e.g. Potholes, Garbage & Sanitation, Drainage & Sewage) from BMC's complaint type/name.
7. Compute res_days for Closed complaints; blank negative or >400-day values.

8. Write complaints.csv (96,999 rows, ~13 MB), a simplified wards.geojson (31 KB) and prepare_stats.json.

Cleaned data summary: 96,999 rows dated 2026-03-18 to 2026-10-11. Categories: Garbage & Sanitation 16,056; Buildings, Licences & Property 15,550; Potholes 12,692; Encroachment & Hawkers 11,519; Drainage & Sewage 10,197; Trees & Gardens 7,607; Roads & Footpaths 7,410; Water Supply 7,194; Pests, Health & Environment 6,939; Street Lights 1,592; Other 243. Status: Closed 76,576; In Progress 14,282; In Review 6,126; Discarded 14; New 1.

2.3 Application architecture

The application is 100% static. The cleaned CSV and ward GeoJSON are served from web/public/data/; the browser loads them (PapaParse), keeps all rows in memory, and runs filters and aggregations in TypeScript. There is no server, database or API key, so it can be deployed to a static host such as Vercel.



2.4 Main modules

- **MapView.tsx —** Three modes: supercluster-based clustered markers, leaflet.heat heatmap, and ward choropleth (count, per 100k residents, avg resolution days, % resolved or the current query result).
- **Filters.tsx —** Category, status, ward, date range, resolution delay ≥ N days and complaint-type text search.
- **QueryPanel.tsx / query.ts —** Query builder (group by, split by, metric, Top-N, minimum group size, order, hotspot rule), generated SQL, Recharts chart and table, CSV export.
- **presets.ts —** Eight preset queries.

3. Implementation

3.1 Categorisation in the cleaning pipeline (scripts/prepare_data.py)

```
def category(row):
    t = str(row["type"] or "").strip(); n = str(row["name"] or "").lower()
    if "pothole" in n: return "Potholes"
    if "street light" in n: return "Street Lights"
    if t in ("Roads and traffic", "Bridges"): return "Roads & Footpaths"
    if t in ("Solid Waste Management",): return "Garbage & Sanitation"
    if t in ("Drainage", "Sewerage Operation", "Storm water drain"): return "Drainage &
Sewage"
    if t == "Water supply": return "Water Supply"
    if t in ("Encroachment", "Hawkers"): return "Encroachment & Hawkers"
    ...
    return "Other"
```

3.2 Removing placeholder coordinates and point-in-polygon ward fill

```
PLACEHOLDER = (19.07766, 72.87741)
ph = (df.lat.round(5) == PLACEHOLDER[0]) & (df.lon.round(5) == PLACEHOLDER[1])
stats["dropped_placeholder_coordinate"] = int(ph.sum())      # 14,546
df = df[~ph].copy()
valid = set(codes)
df["ward"] = df.ward_raw.where(df.ward_raw.isin(valid))
miss = df.ward.isna()                                        # 58 rows
df.loc[miss, "ward"] = [ward_of(a, b) for a, b in zip(df.lat[miss], df.lon[miss])]
df["res_days"] = ((df.resolved - df.created).dt.total_seconds() / 86400).round(2)
df.loc[(df.res_days < 0) | (df.res_days > 400), "res_days"] = pd.NA
```

3.3 Loading the data in the browser (web/src/data.ts)

```
const [csv, geo] = await Promise.all([
  fetch(`${base}data/complaints.csv`).then((r) => r.text()),
  fetch(`${base}data/wards.geojson`).then((r) => r.json()),
]);
const parsed = Papa.parse<Record<string, string>>(csv, { header: true, skipEmptyLines: true
});
const rows: Complaint[] = parsed.data.map((r) => ({
  id: r.id, date: r.date, month: r.date.slice(0, 7), ward: r.ward, status: r.status,
  lat: +r.lat, lon: +r.lon, resDays: r.res_days === "" ? null : +r.res_days, ... }));
```

3.4 Filtering and group-by aggregation (web/src/query.ts)

```
export function applyFilters(rows: Complaint[], f: Filters): Complaint[] {
  const cats = f.categories.length ? new Set(f.categories) : null;
  ...
  return rows.filter((r) =>
    (!cats || cats.has(r.category)) && (!wards || wards.has(r.ward)) &&
    (!f.from || r.date >= f.from) && (!f.to || r.date <= f.to) &&
    (!f.minDelay || (r.resDays !== null && r.resDays >= f.minDelay)) &&
    (!q || r.subcategory.toLowerCase().includes(q)));
}
function value(a: Acc, m: Metric, pop?: number): number | null {
  switch (m) {
    case "count": return a.n;
    case "avg_res_days": return a.k ? +(a.sum / a.k).toFixed(1) : null;
    case "pct_resolved": return a.n ? +((100 * a.closed) / a.n).toFixed(1) : null;
    case "per_100k": return pop ? +((a.n * 100000) / pop).toFixed(1) : null;
  }
}
```

runQuery() makes a single pass over the filtered rows, accumulating counts, closed counts and resolution-day sums per group (and per split group), then applies HAVING (minimum group size), the hotspot rule, ordering and Top-N.

3.5 Hotspot rule

```
if (s.hotspot) {
  const vals = out.map((o) => o.value).filter((v): v is number => v !== null);
  const mean = vals.reduce((x, y) => x + y, 0) / (vals.length || 1);
  const sd = Math.sqrt(vals.reduce((x, y) => x + (y - mean) ** 2, 0) / (vals.length || 1));
  const thr = mean + 0.5 * sd;
```

```
  out.forEach((o) => (o.hot = o.value !== null && o.value >= thr));
  out = out.filter((o) => o.hot);
}
```

3.6 Clustered markers with supercluster (web/src/components/MapView.tsx)

```
const sc = new Supercluster<{ i: number }>({ radius: 55, maxZoom: 17 });
sc.load(rows.map((r, i) => ({ type: "Feature", properties: { i },
  geometry: { type: "Point", coordinates: [r.lon, r.lat] } })));
...
const items = index.getClusters([b.getWest(), b.getSouth(), b.getEast(), b.getNorth()],
                                 Math.round(map.getZoom()));
// heatmap layer
const layer = (L as any).heatLayer(pts, { radius: 12, blur: 16, minOpacity: 0.2, ... });
```

3.7 The eight preset queries (web/src/presets.ts)

Each preset is a QuerySpec; the app generates the equivalent SQL (toSql). Representative SQL text:

1. Top 10 wards by potholes

```
SELECT ward AS ward, COUNT(*) AS count
FROM   complaints
WHERE  category IN ('Potholes')
GROUP BY ward
ORDER BY count DESC
LIMIT 10;
```

2. Monthly trend of potholes

```
SELECT SUBSTR(date, 1, 7) AS month, COUNT(*) AS count
FROM   complaints
WHERE  category IN ('Potholes')
GROUP BY SUBSTR(date, 1, 7)
ORDER BY SUBSTR(date, 1, 7);
```

3. Slowest-resolved categories

```
SELECT category AS category, ROUND(AVG(res_days), 1) AS avg_res_days,
       COUNT(*) AS complaints
FROM   complaints
GROUP BY category
HAVING COUNT(*) >= 100
ORDER BY avg_res_days DESC;
```

4. Hotspot wards (per 100k residents)

```
SELECT ward AS ward, ROUND(COUNT(*) * 100000.0 / w.population, 1) AS per_100k,
       COUNT(*) AS complaints
FROM   complaints c JOIN wards w ON w.ward = c.ward
GROUP BY ward
HAVING per_100k >= (SELECT AVG(per_100k) + 0.5 * STDDEV(per_100k) FROM per_group)
ORDER BY per_100k DESC;
```

5. Category mix per ward (stacked)

```
SELECT ward AS ward, category AS category, COUNT(*) AS count
FROM   complaints
GROUP BY ward, category
ORDER BY count DESC
LIMIT 12;
```

6. Wards with lowest % resolved

```
SELECT ward AS ward,
       ROUND(100.0 * SUM(CASE WHEN status = 'Closed' THEN 1 ELSE 0 END) / COUNT(*), 1)
         AS pct_resolved, COUNT(*) AS complaints
FROM   complaints
GROUP BY ward
HAVING COUNT(*) >= 200
ORDER BY pct_resolved ASC
LIMIT 10;
```

7. Garbage complaints by weekday

```
SELECT STRFTIME('%w', date) AS weekday, COUNT(*) AS count
FROM   complaints
WHERE  category IN ('Garbage & Sanitation')
GROUP BY STRFTIME('%w', date)
ORDER BY STRFTIME('%w', date);
```

8. Water supply complaints per 100k

```
SELECT ward AS ward, ROUND(COUNT(*) * 100000.0 / w.population, 1) AS per_100k,
       COUNT(*) AS complaints
FROM   complaints c JOIN wards w ON w.ward = c.ward
WHERE  category IN ('Water Supply')
GROUP BY ward
ORDER BY per_100k DESC
LIMIT 10;
```

Query 3 (slowest categories) and queries 1, 2, 4 and 5 are shown with their real output in Section 4.

Technologies Used

- **React 19 + Vite + TypeScript —** Single-page application, type-safe data model, fast build.
- **Leaflet / react-leaflet + OpenStreetMap —** Map rendering with free OSM tiles (no API key).
- **supercluster —** Fast clustering of ~97,000 points for the marker view.
- **leaflet.heat —** Heatmap layer of complaint density.
- **Recharts —** Bar, stacked bar and line charts for query results.
- **PapaParse —** In-browser CSV parsing.
- **Python 3, pandas, Shapely —** Download and cleaning pipeline, point-in-polygon ward assignment.
- **BMC MARG ArcGIS REST service —** Source of complaints, potholes and ward boundaries.
- **Vercel (planned) —** Static hosting; root directory web, no environment variables.

4. Results

The summary cards of the app report, for all 96,999 cleaned complaints: 78.9% resolved (status Closed), 20,409 still open (New / In Review / In Progress), an average of 23.9 days to close, Ward KW as the busiest ward (6,889 complaints) and Garbage & Sanitation as the top category (16,056). Screenshots of the running application follow, one under another for readability.

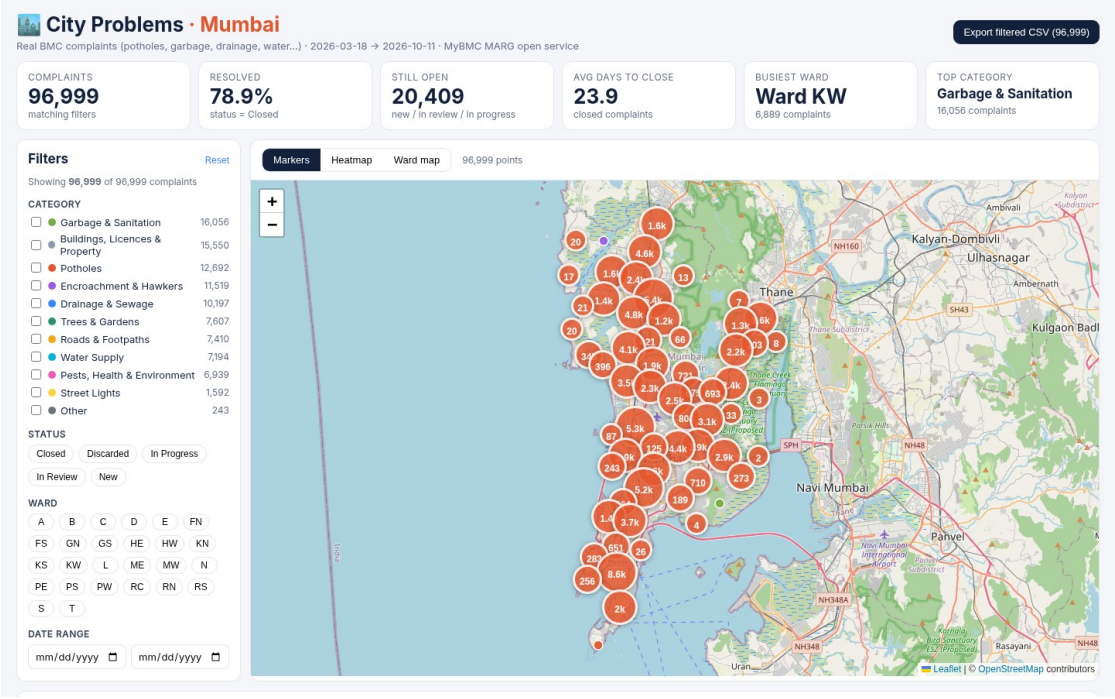


Fig. 1: Map – clustered markers of 96,999 complaints with filters and summary cards

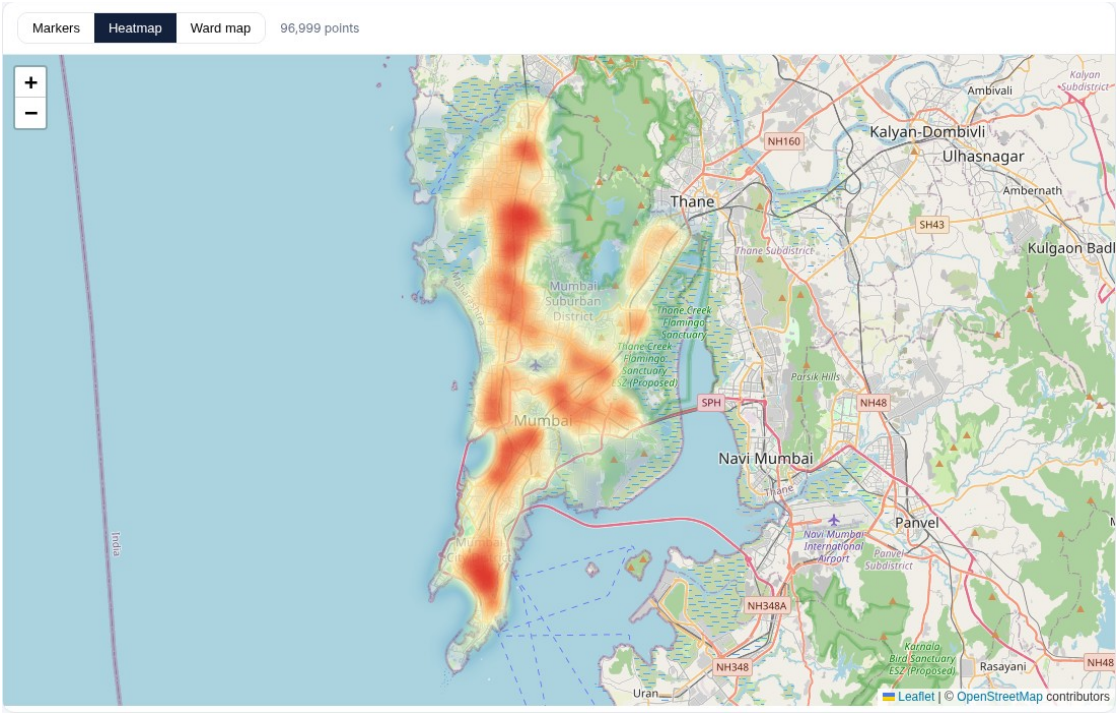


Fig. 2: Map – heatmap showing complaint density across Mumbai

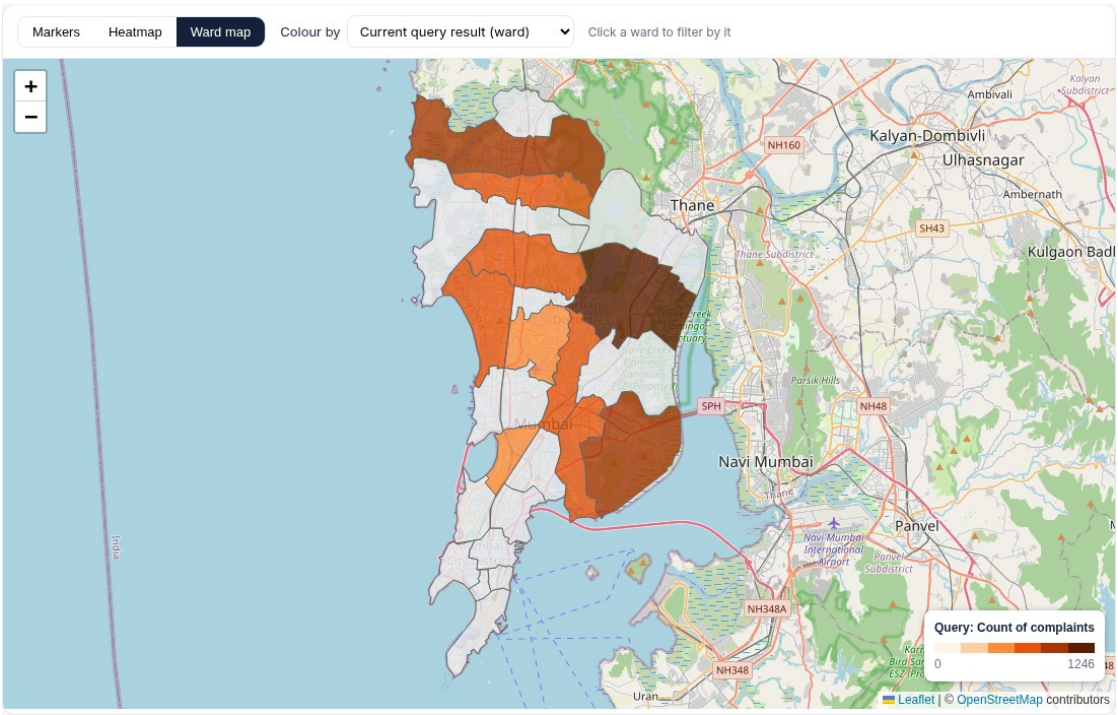


Fig. 3: Ward choropleth coloured by the current query result (complaint count)

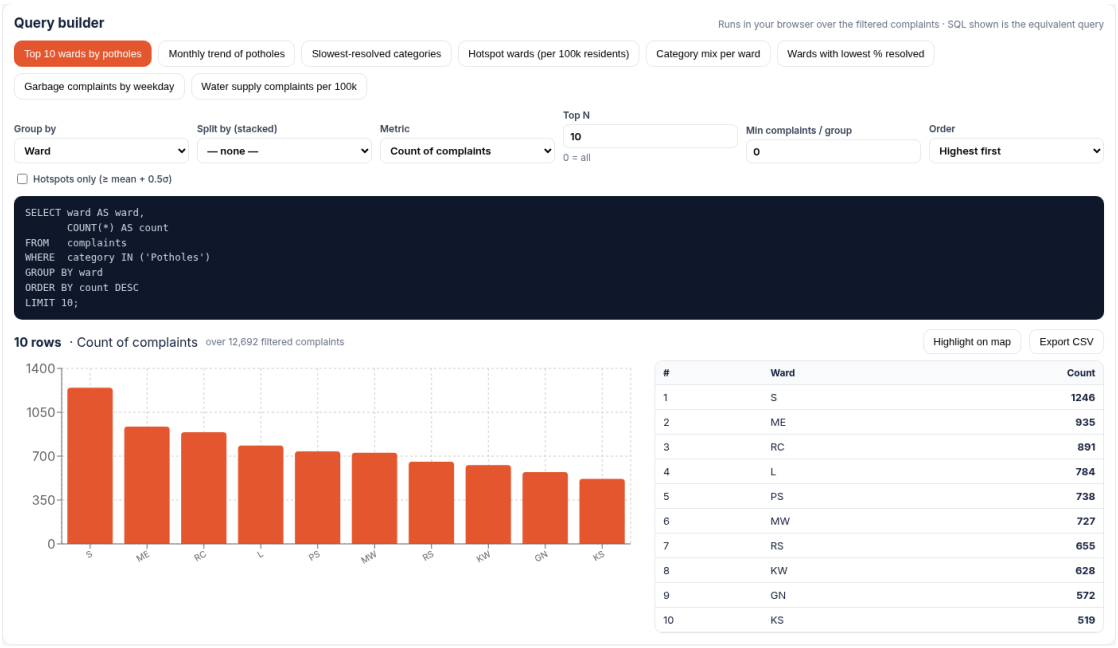


Fig. 4: Query 1 – Top 10 wards by potholes (ward S highest with 1,246 of 12,692 pothole complaints), with generated SQL, chart and table

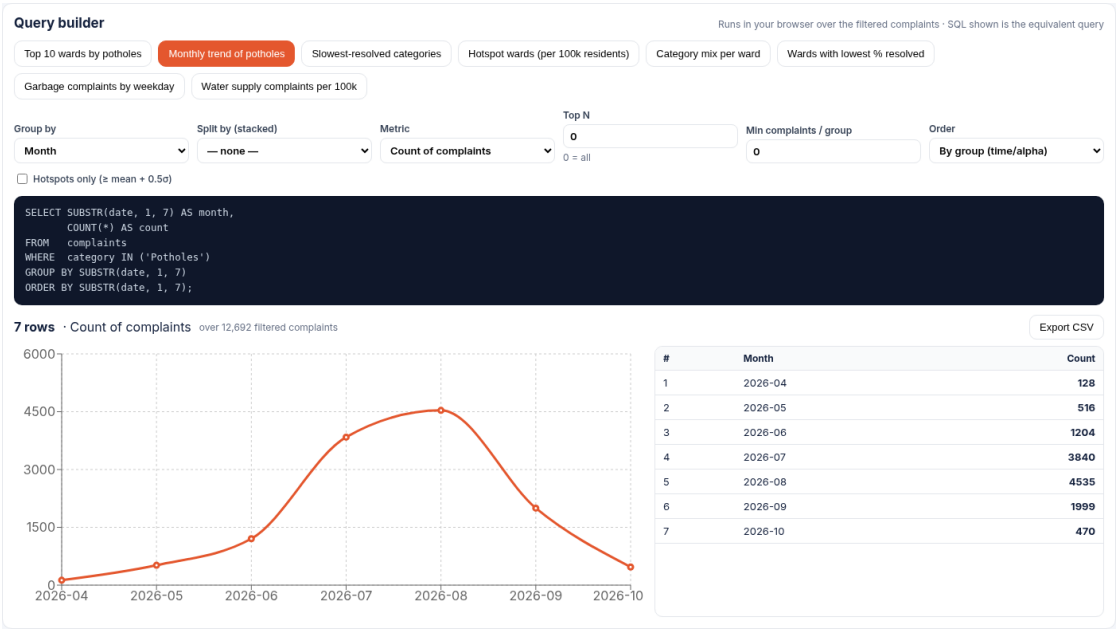


Fig. 5: Query 2 – Monthly pothole trend: complaints rise from 128 (Apr) to a peak of 4,535 (Aug) and fall to 1,999 (Sep), consistent with the monsoon (Oct is a partial month)

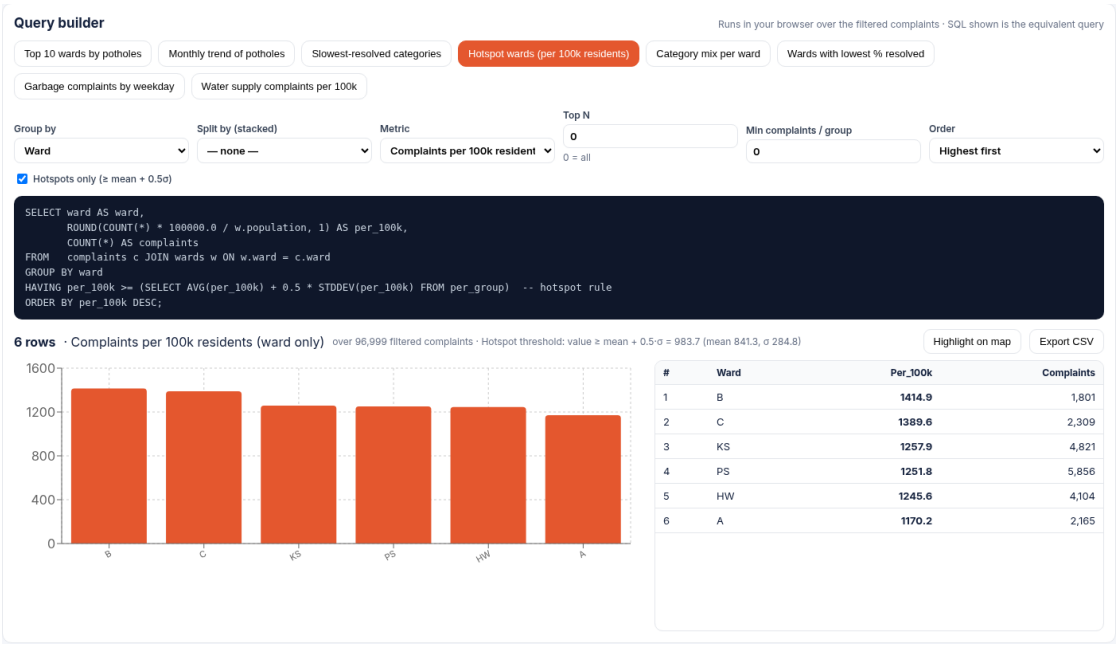


Fig. 6: Query 4 – Hotspot wards per 100k residents (threshold 983.7): B, C, KS, PS, HW and A

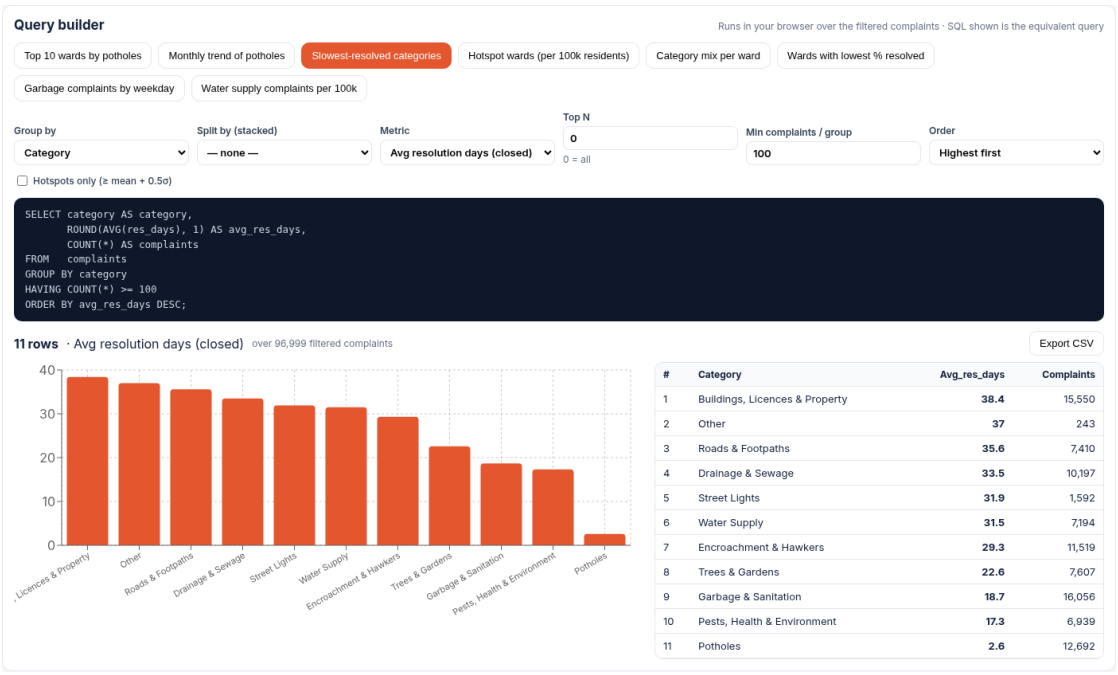


Fig. 7: Query 3 – Slowest-resolved categories: Buildings, Licences & Property average 38.4 days; Potholes only 2.6 days

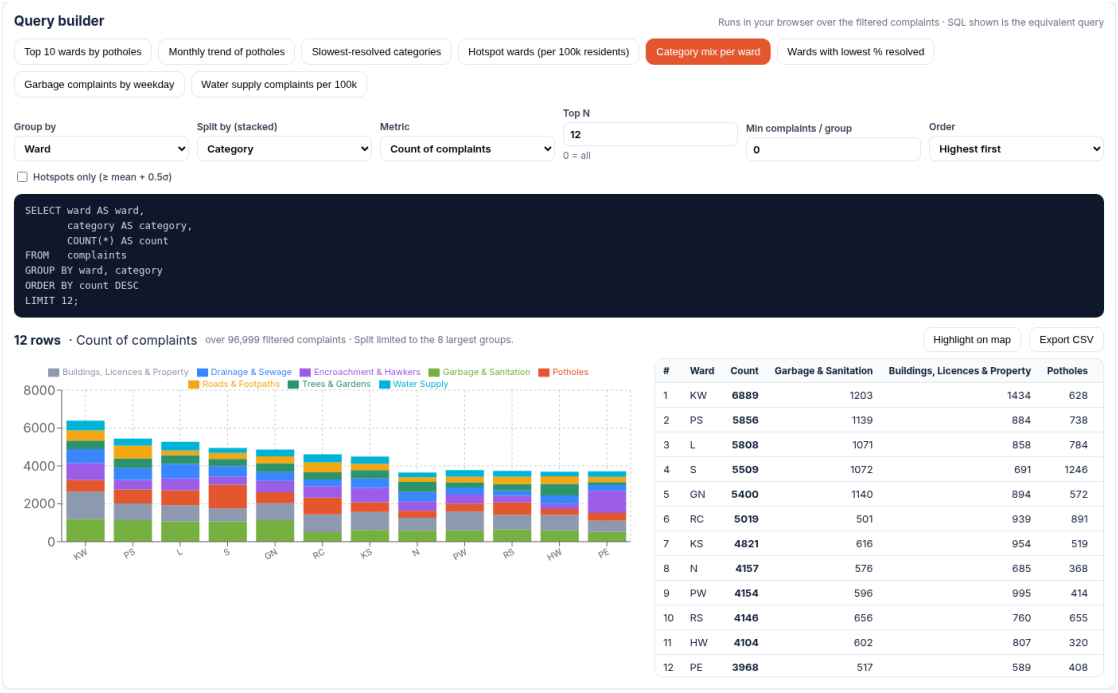


Fig. 8: Query 5 – Category mix for the 12 busiest wards (stacked), led by KW with 6,889 complaints

Observations

- Complaints concentrate in the western and central suburbs; the heatmap also shows a strong cluster at the southern tip of the city.
- Pothole complaints follow the monsoon: they grow from April and peak in August.
- Potholes are closed far faster (2.6 days on average) than building/licence or road-related complaints (35–38 days).
- Per-capita rates change the picture: wards B, C, KS, PS, HW and A are hotspots per 100k residents even if raw counts differ.

Limitations and caveats

- The data is a single ~7-month snapshot (March–October 2026), not a multi-year history; October 2026 is a partial month.
- Complaints are not the same as problems: busier wards file more, and filing behaviour differs by channel.
- Resolution days: BMC publishes no true closure date for general complaints, so for them resolution days is the number of days from the complaint date to the date of the “after” photo. For potholes, RESOLVED_DATE is used.
- Severity is not published; the “resolution delay ≥ N days” filter is only a proxy.
- Removing 14,546 placeholder-coordinate rows also removes almost all Discarded/invalid complaints from the map data.
- No explicit BMC licence: data is used for non-commercial academic purposes with attribution.

5. Conclusion

City Problems demonstrates a complete small-scale big-data workflow on real civic data: acquisition from a public ArcGIS service, cleaning and integration of 124,146 raw records into 96,999 reliable geo-tagged complaints, and an interactive map and query workbench that performs group-by, aggregate and per-capita analytics over the full dataset in the browser. The eight preset queries answer practical questions about where potholes and other issues concentrate, when they peak and how quickly they are resolved.

The data-quality work was essential: a single placeholder coordinate would otherwise have distorted the ward-level results. Future work includes multi-year data, a real closure timestamp, severity estimation, a server-side store for larger volumes, and deployment on Vercel as a static site.

6. References

1. Brihanmumbai Municipal Corporation (MCGM), MARG ArcGIS REST service, https://marg.mcgm.gov.in/server/rest/services/MARG_APP/MARG_Application_Prod/MapServer (accessed 11 Oct 2026).

2. BMC Pothole QuickFix Dashboard, https://marg.mcgm.gov.in/BMC_C_Pothole_Dashboard/ (accessed 2026).

3. Leaflet – an open-source JavaScript library for interactive maps, https://leafletjs.com/.

4. OpenStreetMap contributors, map tiles and data, https://www.openstreetmap.org/copyright.

5. V. Agafonkin, supercluster – a fast geospatial point clustering library, https://github.com/mapbox/supercluster.

6. Leaflet.heat – a tiny, simple and fast heatmap plugin for Leaflet, https://github.com/Leaflet/Leaflet.heat.

7. Recharts – a composable charting library for React, https://recharts.org/.

8. React Documentation, https://react.dev/; Vite Documentation, https://vite.dev/.

9. S. Gillies et al., Shapely – manipulation and analysis of geometric objects, https://shapely.readthedocs.io/.

10. W. McKinney, “Data Structures for Statistical Computing in Python,” Proc. 9th Python in Science Conf., 2010 (pandas).

11. Shree L. R. Tiwari College Of Engineering, Department of Computer Engineering – BDA Mini Project guidelines (internal).